Note: the second reward function provides more credit to the response than the first reward function.

⇓

capture the structurally sorting progress, not the complete matching

Design reward function is more important that ~~the~~ policy design

Def simple_model():

define a simple model that maps prompts to responses

① Assume fixed prompt and response length.

② captures positional information with separate per-position parameters

③ Decode each position in the response independently

① ⇒ define a model (not Auto-regressive)

model = Model(vocab_size=3, embedig_dim=16, prompt_length=3, response_length=3)

② ⇒ Start with a prompt: 5 batch 3 tokens

prompts = torch.tensor([[0, 1, 2]]): #[batch pos]

③ ⇒ Generate a response

torch.manual_seed(0) → return: [batch trial #tokens pos]

response = generate_responses(prompts, model, num_response)

prompt trajectory batch size prompt length

How many tokens

trial: 多条 trajectory

④ ⇒ [return [batch Trial]]

compute Rewards R of these responses:

rewards = compute_reward(prompts, responses, ~~reward_fn~~ score_inclusion_ordering_reward)

⑤ ⇒ compute deltas - 5: normalization for ~~reward~~ Advantage/rewards: given the Rewards

deltas = centered_rewards, normalized_rewards, map_rewards

↖ multiple calculations method